

8-Point Radix-2 DIF SDF FFT — MATLAB Modeling Plan

0. What You Are Ultimately Building

Build a **hardware-oriented MATLAB model** of an:

- 8-point FFT
- Radix-2
- DIF (Decimation in Frequency)
- SDF (Single-path Delay Feedback)

For $N = 8$:

- Number of stages $= \log_2(8) = 3$
- Stage 1 $\rightarrow$ delay 4
- Stage 2 $\rightarrow$ delay 2
- Stage 3 $\rightarrow$ delay 1

```
Input
|
v
Stage 1 — Delay 4
|
v
Stage 2 — Delay 2
|
v
Stage 3 — Delay 1
|
v
FFT Output
```

The goal is **not** simply to calculate an FFT with MATLAB. The goal is to model the actual hardware architecture and understand its cycle-by-cycle data movement.

PHASE 1 — Establish the Mathematical Reference

Step 1 — Define FFT Parameters

Use:

$N = 8$

Therefore:

$\log_2(8) = 3$ stages

Delay lengths:

Stage 1 → 4
Stage 2 → 2
Stage 3 → 1

Define:

$x[0], x[1], \dots, x[7]$

Start with simple known input values.

Step 2 — Understand the Ordinary 8-Point DIF FFT

Before modeling SDF, understand the normal Radix-2 DIF flow graph.

Identify which samples are paired at each stage.

Do not introduce feedback or hardware timing yet.

PHASE 2 — Understand the DIF Butterfly

Step 3 — Model the DIF Butterfly

The basic operation is:

SUM = A + B
DIFF = A - B
TWIDDLED_DIFF = DIFF * W

Conceptually:

A ---->--- DIF Butterfly --- SUM
B -----/ DIFF × W

Understand the butterfly independently before introducing delays.

PHASE 3 — Build the Twiddle-Factor Model

Step 4 — Generate W_8^k

Use:

$W_8^k = \exp(-j \cdot 2 \cdot \pi \cdot k / 8)$

Generate the twiddle factors from N and k rather than manually entering decimal values.

Step 5 — Create a Twiddle Table

Create a table containing:

k W^k **Real** **Imaginary**

0 W^0

1 W^1

2 W^2

3 W^3

Determine which twiddles are required by each stage.

PHASE 4 — Build the Delay Model

Step 6 — Build a Generic Delay Line

Temporarily forget the FFT.

Model a delay-4 shift register.

For example:

cycle	input	delayed output
0	x0	?
1	x1	?
2	x2	?
3	x3	?
4	x4	x0
5	x5	x1
6	x6	x2
7	x7	x3

Make the exact delay convention explicit.

Goal:

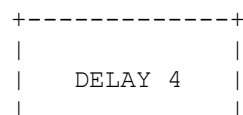
At every clock, one new sample enters and the oldest sample leaves.

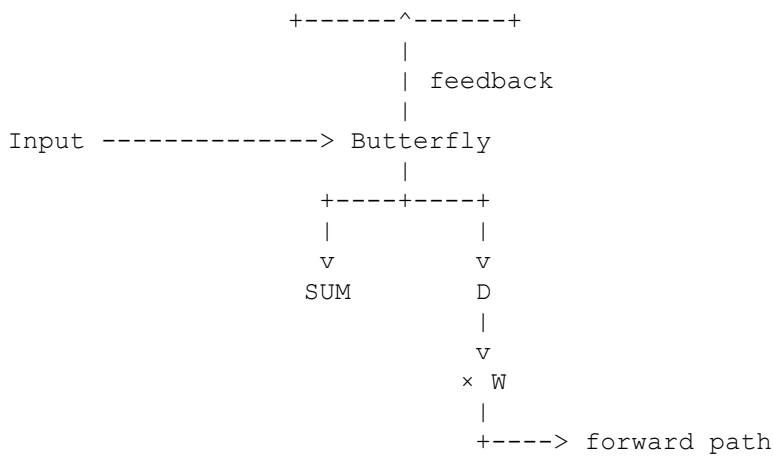
PHASE 5 — Understand SDF Feedback

Step 7 — Analyze the Feedback Before Coding

The SDF idea is to use one delay module with feedback rather than the two delay paths used in a simple pipeline.

Conceptually:





Before coding, answer:

1. What is stored?
2. What comes out of the delay?
3. When does the butterfly operate?
4. Where does $A+B$ go?
5. Where does $A-B$ go?
6. Where is twiddle multiplication?
7. What comes back through feedback?
8. What goes to the next stage?

PHASE 6 — Build ONLY Stage 1

Step 8 — Create Stage 1

Use:

- $N = 8$
- Delay = 4

Do not build Stages 2 and 3 yet.

Stage 1 is the most important milestone.

Step 9 — Make Stage 1 Cycle-by-Cycle

Model the architecture as clocked hardware:

```

cycle 0 → input x0
cycle 1 → input x1
cycle 2 → input x2
...
cycle 7 → input x7

```

Track every cycle:

- input sample
- delay contents

- delay output
- butterfly enable
- butterfly inputs
- sum
- difference
- twiddle index
- twiddle value
- feedback value
- forward value
- valid

PHASE 7 — Create a Stage-1 Trace

Step 10 — Print a Cycle Trace

Create a trace similar to:

Cycle	Input	Delay Out	BF Enable	A	B	Sum	Diff	W	Output
0									
1									
2									
...									

The goal is to make the architecture visible during simulation.

PHASE 8 — Verify Stage 1

Step 11 — Verify Arithmetic

For each butterfly:

$$\text{SUM} = A + B$$

$$D = (A - B) * W$$

Compare MATLAB results against manual calculations.

Step 12 — Verify the Delay

Verify:

input → delay → output

Make sure the delay output is exactly the sample expected for that cycle.

If Stage 1 fails, identify whether the issue is:

- delay

- butterfly
 - twiddle
 - feedback scheduling
-

PHASE 9 — Build Stage 2

Step 13 — Create Stage 2

Feed Stage 1 output into:

Stage 2, Delay = 2

Determine:

- when the delay is filling
- when the butterfly is active
- which data pairs
- which twiddle is selected
- what goes forward
- what goes through feedback

Again, create a timing table.

PHASE 10 — Build Stage 3

Step 14 — Create Stage 3

Use:

Stage 3, Delay = 1

Repeat:

```
Determine timing
    ↓
Determine delay behavior
    ↓
Determine butterfly pairs
    ↓
Determine twiddles
    ↓
Determine feedback
    ↓
Create MATLAB model
    ↓
Create trace
    ↓
Verify
```

PHASE 11 — Connect the Three Stages

Step 15 — Connect

```
Input
|
v
Stage 1 - Delay 4
|
v
Stage 2 - Delay 2
|
v
Stage 3 - Delay 1
|
v
Output
```

At this point the functional SDF architecture exists.

PHASE 12 — Understand Latency

Step 16 — Determine Output Timing

Do not only ask whether the final FFT values are correct.

Also determine:

At which clock does each output appear?

Track:

```
input clock
  ↓
stage 1 latency
  ↓
stage 2 latency
  ↓
stage 3 latency
  ↓
output clock
```

This will be important for RTL.

PHASE 13 — Add Valid Signals

Step 17 — Introduce Valid

Eventually model:

```
input_data
input_valid
```

and:

```
output_data
output_valid
```

Distinguish between:

- data existing internally
- output data being valid

This will be important in SystemVerilog.

PHASE 14 — Check DIF Output Ordering

Step 18 — Handle Output Ordering

DIF has a different natural output ordering.

Determine the ordering produced by the architecture and explicitly model the required output permutation / bit reversal.

Keep these concepts separate:

```
SDF raw output
```

and:

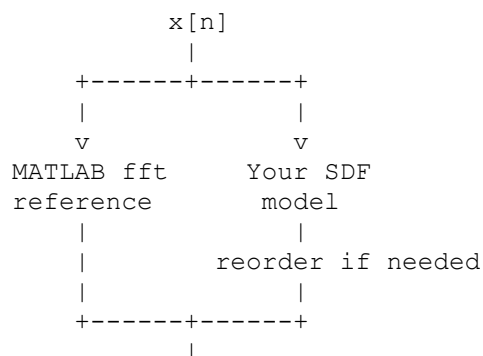
```
correctly ordered FFT output
```

PHASE 15 — Compare With MATLAB `fft()`

Step 19 — Create the Golden Reference

Use MATLAB's built-in `fft()` **only as a reference**.

Conceptually:



v
Compare

Do NOT use `fft()` internally to implement the SDF FFT.

PHASE 16 — Verify Internal Checkpoints

Step 20 — Verify Every Stage

Create checkpoints:

```
Input
|
v
Stage 1 --> checkpoint 1
|
v
Stage 2 --> checkpoint 2
|
v
Stage 3 --> checkpoint 3
|
v
Output --> checkpoint 4
```

If the final output is wrong, find the first stage where the error appears.

PHASE 17 — Test Different Inputs

Step 21 — Simple Sequence

0 1 2 3 4 5 6 7

Step 22 — Impulse

1 0 0 0 0 0 0 0

Step 23 — Constant

1 1 1 1 1 1 1 1

Step 24 — Complex Input

Use:

$x[n] = a[n] + j*b[n]$

Step 25 — Random Input

Only after all previous tests work.

PHASE 18 — Verify Individual Building Blocks

Before declaring the complete FFT correct:

```
Twiddle generator -----> ✓
Butterfly -----> ✓
Delay 4 -----> ✓
Stage 1 -----> ✓
Stage 2 -----> ✓
Stage 3 -----> ✓
Full SDF -----> ✓
Output ordering -----> ✓
Latency -----> ✓
Final FFT -----> ✓
```

PHASE 19 — Move to Fixed Point

Only after the floating-point model works.

Development order:

```
Floating point
  ↓
Functional SDF
  ↓
Cycle-accurate SDF
  ↓
Verify against fft()
  ↓
Fixed-point model
  ↓
Quantization analysis
  ↓
SystemVerilog
```

Do not start with fixed point.

Final MATLAB Project Structure

Eventually:

```
FFT_SDF/
|
+-- main
+-- parameters
+-- twiddle_generator
+-- butterfly_DIF
```

```
+-- delay_line
+-- SDF_stage
+-- SDF_FFT
+-- output_ordering
+-- reference_fft
+-- verification
```

Do not create everything at once. Build each component as you reach it.

Milestones

Milestone 1

Understand and model the DIF butterfly.

Milestone 2

Generate and verify w_8^k .

Milestone 3

Model Delay-4 and prove its cycle-by-cycle behavior.

Milestone 4 ☐

Build Stage 1 SDF, Delay 4.

Milestone 5

Build Stage 2 SDF, Delay 2.

Milestone 6

Build Stage 3 SDF, Delay 1.

Milestone 7

Connect Stage 1 $\rightarrow$ Stage 2 $\rightarrow$ Stage 3.

Milestone 8

Verify SDF output against MATLAB `fft()`, including output ordering.

Milestone 9

Make the model cycle-accurate with data, valid, latency, and stage state.

Milestone 10

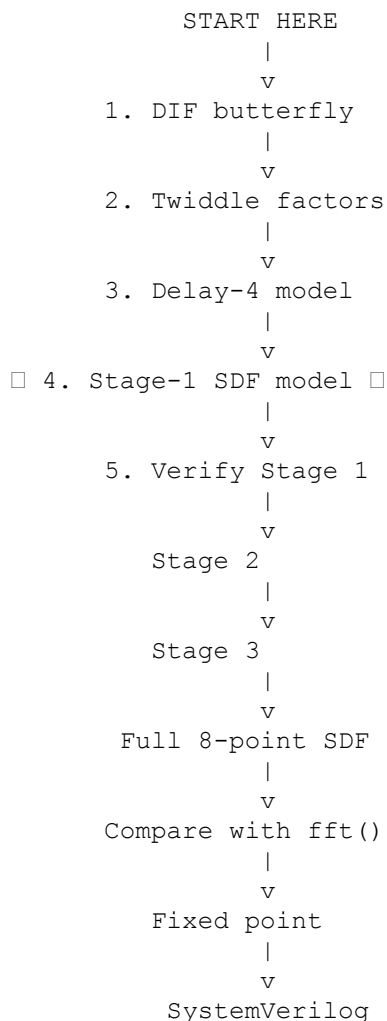
Convert the floating-point model to fixed point and analyze error.

Milestone 11

Move to SystemVerilog RTL.

At that point, the MATLAB model becomes the golden behavioral reference for the RTL.

Immediate Path



Most Important Principle

The difficult part of this project is not MATLAB syntax.

The difficult part is translating:

FFT mathematics

↓

signal-flow graph
↓
SDF architecture
↓
cycle-by-cycle data movement
↓
MATLAB model
↓
SystemVerilog RTL

Keep these layers separate throughout the project.